Sri Lanka Institute of Information Technology



Streetify

Web-Based Transportation System.

Lab Report 06: Software Testing

Group ID : 2026-Y2-S1-KU-24 (Group 24)

IT25102193	Weerasingha W.A.M.B.
IT25102207	Dharmakeerthi W.A.C.B.
IT25102208	Nayanamina A.R.L.P.
IT25102225	Edirisinghe E.A.R.N.D.
IT25102240	Rammandalagedara R.V.S.
IT25102241	Senaka K.A.T.

| SE2030 | Software Engineering | 2026 | 2Y1S |

Group Members & Assigned Modules

Member Name	Registration No.	Assigned Function / Module
A.R.L.P. Nayanamina (Lahiru)	**IT25102208**	User Account Management
W.A.C.B. Dharmakeerthi (Chanuka)	**IT25102207**	Ride Booking and Management
K.A.T. Senaka (Tharindu)	**IT25102241**	Driver Trip Request Management
E.A.R.N.D. Edirisinghe (Daham)	**IT25102225**	Payment and Receipt Management
W.A.M.B. Weerasingha (Mithun)	**IT25102193**	Review, Rating, and Feedback Management
R.V.S. Rammandalagedara (Vidura)	**IT25102240**	Administration and System Control

Group Details & Execution Summary

Group ID	2026-Y2-S1-KU-24 (Group 24)
Topic Name	Streetify – Web-Based Transportation System
Module	SE2030 - Software Engineering (Year 2 Semester 1 - 2026)
Testing Result	60 passed, 0 failed (100% Health & Integration Pass Rate)
Test Execution	Executed across full stack (React TS frontend <-> Spring Boot Java backend <-> MSSQL db)
Total Test Cases	60 Test Cases (6 Team Members x 10 Comprehensive Cases Each)

No.	Member Name	Registration No.	Assigned Function	Test Case IDs	Passed
01	Lahiru (A.R.L.P. Nayanamina)	IT25102208	User Account Management	TC_UM_01 - 10	**10 / 10**
02	Chanuka (W.A.C.B. Dharmakeerthi)	IT25102207	Ride Booking and Management	TC_BK_01 - 10	**10 / 10**
03	Tharindu (K.A.T. Senaka)	IT25102241	Driver Trip Request Management	TC_TM_01 - 10	**10 / 10**
04	Daham (E.A.R.N.D. Edirisinghe)	IT25102225	Payment and Receipt Management	TC_PM_01 - 10	**10 / 10**
05	Mithun (W.A.M.B. Weerasingha)	IT25102193	Review, Rating, and Feedback Management	TC_RM_01 - 10	**10 / 10**
06	Vidura (R.V.S. Rammandalagedara)	IT25102240	Administration and System Control	TC_AM_01 - 10	**10 / 10**

Validation Strategy & Layered System Integration

Each team member created 10 industry-grade test cases for their assigned module and executed them on Streetify's live multi-tier system. The stack included a React (TypeScript + Tailwind CSS) frontend, a Spring Boot (Java 24) REST backend with Bean Validation, and an MSSQL database. Test results captured HTTP status codes, JSON responses, and database state checks.

- **Positive tests:** validated normal operations across the system, such as passenger signup, fare calculation, driver status updates, and payment settlement.
- **Negative tests:** checked invalid or malformed inputs to ensure both frontend and backend rejected unauthorized actions with proper error feedback.
- **Boundary value tests:** examined inputs at and beyond system thresholds, including password lengths, star ratings, wait timers, wallet balances, and audit log limits.
- **Role-Based Access Control (RBAC) & Security tests:** confirmed Spring Security rules enforced role restrictions, returning HTTP 403 Forbidden for users lacking required roles.

Test Case ID format follows TC_<module>_<number>

- UM = User Management
- BK = Ride Booking
- TM = Trip Management & Telemetry
- PM = Payment Management
- RM = Review & Disputes
- AM = Admin Control)

1. User Account Management

Member: A.R.L.P. Nayanamina (Lahiru) | **Registration No.: IT25102208**

Scope: Passenger & Driver Registration, JWT Authentication, Login Lockout & Security, Email & Sri Lankan Phone Validations, BCrypt Password Security, Password Reset with 5-minute OTP.

Test Case ID	Test Title	Description	Preconditions	Test Steps	Test Data	Expected Output	Actual Output	Status
TC_UM_01	Verify passenger registration with valid details	Ensure a new passenger can create an account with complete valid inputs.	1. User is not logged in 2. Email is not previously registered	1. Open Register page 2. Enter name, email, phone, and 8+ char password 3. Click 'Create Account'	Name: Kasun Perera Email: kasun.p@streetify.lk Phone: +94771234567 Password: Pass#Secure2026	HTTP 201 Created; JWT tokens issued; user saved with role PASSENGER in MSSQL database.	Account created (HTTP 201); status ACTIVE; JWT access token generated; passenger redirected to dashboard.	**Pass**
TC_UM_02	Verify registration with existing duplicate email	Ensure the system prevents duplicate registrations with the same email.	1. Account with email passenger1@streetify.com already exists in database	1. Open Register page 2. Enter details using duplicate email 3. Click 'Create Account'	Email: passenger1@streetify.com Name: Duplicate Tester Password: Pass#Secure2026	Registration rejected with HTTP 400 Bad Request; clear duplicate email error message.	Rejected (HTTP 400) with 'Email is already taken by another account.' Account not created.	**Pass**
TC_UM_03	Verify password with 7 characters (below minimum)	Boundary test: passwords must be at least 8 characters long.	1. User is on Register page 2. Email is unregistered	1. Enter valid name, email, and phone 2. Enter a 7-character password 3. Click 'Create Account'	Password: Pass#12 (7 chars) Email: test7char@streetify.lk	Registration blocked; validation error specifies minimum 8 characters.	Rejected (HTTP 400) on password field: 'Password must be at least 8 characters'.	**Pass**
TC_UM_04	Verify password with exactly 8 characters (minimum allowed)	Boundary test: verify shortest valid password length (8 characters) is accepted.	1. User is on Register page 2. Email is unregistered	1. Enter valid personal details 2. Enter exactly 8-character password 3. Click 'Create Account'	Password: Pass#123 (8 chars) Email: test8char@streetify.lk	Registration succeeds; password BCrypt hashed; JWT returned.	Account created successfully (HTTP 201); password encrypted; JWT tokens generated.	**Pass**
TC_UM_05	Verify email format without '@' symbol	Ensure email format enforces presence of '@' symbol.	1. User is on Register or Profile screen	1. Enter an email missing the '@' sign 2. Submit registration/profile form	Email: invalidemail.streetify.lk Name: Test User	Validation error displayed: email must be a valid email containing '@'.	Rejected (HTTP 400) with 'Email must be a valid email address containing the "@" symbol (e.g. user@streetify.lk)'.	**Pass**
TC_UM_06	Verify driver registration with valid Sri Lankan phone format	Ensure driver registration accepts standard Sri Lankan phone with '+94' prefix.	1. User on driver onboarding form 2. NIC and vehicle details provided	1. Enter driver personal and vehicle details 2. Enter phone with '+94' prefix 3. Submit onboarding form	Phone: +94779876543 NIC: 951234567V Vehicle: Toyota Prius (WP CAB-1234) Year: 2018	Driver account created with status PENDING_VERIFICATION (HTTP 201).	Driver registered (HTTP 201); verification_status set to PENDING_VERIFICATION in MSSQL.	**Pass**

TC_UM_07	Verify driver registration with invalid phone format	Ensure driver phone format strictly enforces '+94' or '0' followed by 9 digits.	1. User on driver onboarding form	1. Enter driver details 2. Enter 8-digit phone number without valid prefix 3. Click 'Register as Driver'	Phone: 12345678 (8 digits, invalid prefix) Email: driver.test@streetify.lk	Validation rejected; message explains valid Sri Lankan phone formats.	Rejected (HTTP 400): 'Driver phone number must be either "+94" followed by 9 digits or "0" followed by 9 digits'.	**Pass**
TC_UM_08	Verify user login with valid credentials	Ensure registered user can authenticate and obtain JWT access token.	1. Account passenger1@streetify.com is active in database	1. Open Login screen 2. Enter registered email and password 3. Click 'Sign In'	Email: passenger1@streetify.com Password: 1111	HTTP 200 OK; JWT access token & user role PASSENGER returned; redirect to home.	Login successful (HTTP 200); JWT issued with claims; role PASSENGER loaded in state.	**Pass**
TC_UM_09	Verify login with incorrect password	Ensure login fails securely on incorrect password without leaking data.	1. Account passenger1@streetify.com is registered	1. Open Login screen 2. Enter correct email but incorrect password 3. Click 'Sign In'	Email: passenger1@streetify.com Password: WrongPassword#999	Login refused with HTTP 401 Unauthorized; message 'Bad credentials'.	Refused (HTTP 401) with 'Bad credentials'. No JWT issued; login denied.	**Pass**
TC_UM_10	Verify password reset OTP verification within 5-minute expiry	Ensure 6-digit OTP allows password update within its 5-minute validity window.	1. Account kasun@streetify.com exists 2. OTP requested within last 5 minutes	1. Request OTP for registered email 2. Enter the generated 6-digit OTP 3. Enter new 8-char password and submit	Email: kasun@streetify.com OTP: 6-digit code New Password: NewPass#2026	OTP validated; password hash updated in MSSQL; HTTP 200 message returned.	Verified (HTTP 200); password hash updated; OTP cache entry invalidated.	**Pass**

2. Ride Booking and Management

Member: W.A.C.B. Dharmakeerthi (Chanuka) | **Registration No.: IT25102207**

Scope: Passenger Ride Booking, Fare Estimation Engine, Geocoding & Coordinate Validation, Vehicle Type Selection, Nearby Driver Dispatch, Trip History, Trip Cancellation.

Test Case ID	Test Title	Description	Preconditions	Test Steps	Test Data	Expected Output	Actual Output	Status
TC_BK_01	Verify fare estimation for valid route	Ensure passenger receives accurate fare breakdown based on route distance.	1. Passenger logged in 2. GPS geocoding service active	1. Open Booking screen 2. Enter pickup and dropoff locations 3. Click 'Get Fare Estimate'	Pickup: Colombo Fort (6.9344, 79.8428) Dropoff: SLIIT Malabe (6.9147, 79.9729) Type: CAR	HTTP 200 OK; returns base fare, per km rate, platform fee, and total estimated fare.	Fare estimate returned: Base LKR 300.00, Total LKR 2,480.00; route line drawn on map.	**Pass**
TC_BK_02	Verify booking a ride with complete valid details	Ensure passenger can request a ride and broadcast request to available drivers.	1. Passenger logged in as passenger1@streetify.com 2. Route selected	1. Select pickup and dropoff points 2. Choose ride type 'CAR' 3. Select payment method 'CARD' 4. Click 'Book Ride'	Pickup: Colombo Fort Dropoff: Nugegoda RideType: CAR Payment: CARD	Trip created with status REQUESTED (HTTP 201); broadcast sent via WebSocket.	Trip #101 created (HTTP 201) with status REQUESTED; trip inserted in MSSQL trips table.	**Pass**
TC_BK_03	Verify booking with missing pickup location	Ensure booking is rejected if pickup address or coordinates are missing.	1. Passenger on Booking screen	1. Leave pickup address and coordinates empty 2. Select dropoff location 3. Click 'Book Ride'	Pickup: null Dropoff: Kottawa Junction RideType: TUK	Request blocked with HTTP 400 Bad Request; pickup address is required.	Rejected (HTTP 400) on pickupAddress: 'Pickup address is required'. No trip created.	**Pass**
TC_BK_04	Verify booking with identical pickup and dropoff coordinates	Boundary test: distance between pickup and dropoff must be at least 0.1 km.	1. Passenger on Booking screen	1. Set pickup coordinates 2. Set identical dropoff coordinates (0.00 km) 3. Click 'Book Ride'	Pickup: (6.9344, 79.8428) Dropoff: (6.9344, 79.8428) Distance: 0.0 km	Request rejected: pickup and dropoff locations cannot be identical.	Rejected (HTTP 400): 'Pickup and dropoff locations cannot be identical (min distance 0.1 km)'.	**Pass**
TC_BK_05	Verify coordinates outside Sri Lanka operational boundary	Boundary test: reject coordinates outside Sri Lanka geographic boundary.	1. Passenger on Booking screen	1. Enter pickup latitude outside Sri Lanka (Lat > 10.0 or Lng > 82.5) 2. Submit booking request	Pickup Lat: 15.5000, Lng: 95.0000 (Sri Lanka boundary: Lat 5.9-9.9, Lng 79.5-81.9)	Request rejected: location is outside Streetify operational service area.	Rejected (HTTP 400): 'Location coordinates are outside the Streetify Sri Lanka operational boundary'.	**Pass**
TC_BK_06	Verify available trips listing for online drivers	Ensure an online driver can query all unassigned REQUESTED trips.	1. Driver driver1@streetify.com logged in 2. Trip #101 in REQUESTED status	1. Open Driver Dispatch dashboard 2. System calls GET /api/rides/available	Driver: driver1@streetify.com Status filter: REQUESTED	HTTP 200 OK; list of unassigned trips with fare and distance returned.	HTTP 200 OK; Trip #101 listed with status REQUESTED, fare LKR 1925.00, route details.	**Pass**

TC_BK_07	Verify driver accepting an available trip	Ensure driver can accept an unassigned trip and update its status to ACCEPTED.	1. Trip #101 is REQUESTED 2. Driver driver1@streetify.com is online	1. Driver clicks 'Accept Trip' on Trip #101 2. System calls POST /api/rides/accept/101	Trip ID: 101 Driver ID: 10 (driver1@streetify.com)	Trip status updated to ACCEPTED; driver assigned in DB; passenger notified.	HTTP 200 OK; trip status updated to ACCEPTED; driver_id set to 10 in MSSQL trips.	**Pass**
TC_BK_08	Verify second driver cannot accept an already accepted trip	Concurrency test: ensure two drivers cannot accept the same trip.	1. Trip #101 already accepted by Driver 1 (status ACCEPTED) 2. Driver 2 is logged in	1. Driver 2 clicks 'Accept Trip' on Trip #101 2. System calls POST /api/rides/accept/101	Trip ID: 101 Driver 2 ID: 11 (driver2@streetify.com)	Request rejected with HTTP 409 Conflict: trip already accepted.	Rejected (HTTP 409) with 'Trip has already been accepted by another driver.'	**Pass**
TC_BK_09	Verify passenger cancelling a requested ride	Ensure passenger can cancel a trip prior to commencement.	1. Passenger has active Trip #102 in REQUESTED status	1. Click 'Cancel Ride' button 2. Confirm cancellation with reason	Trip ID: 102 Reason: 'Change of schedule'	Trip status set to CANCELLED in DB; drivers notified; HTTP 200 returned.	HTTP 200 OK; status updated to CANCELLED in MSSQL; message 'Trip cancelled successfully'.	**Pass**
TC_BK_10	Verify passenger cannot access another user's private trip	Security check: ensure passengers cannot query or modify other users' trips.	1. Passenger A is logged in 2. Trip #103 belongs to Passenger B	1. Passenger A attempts to call cancel/view endpoint for Trip #103	Target Trip ID: 103 (Passenger B) Caller: Passenger A	Access refused with HTTP 403 Forbidden / 404 Not Found.	Refused (HTTP 403) with 'You are not authorized to access or modify this trip.'	**Pass**

3. Driver Trip Request Management

Member: K.A.T. Senaka (Tharindu) | **Registration No.: IT25102241**

Scope: Trip Progress State Machine (EN_ROUTE -> ARRIVED -> IN_PROGRESS -> COMPLETED), Illegal State Transitions, Passenger No-Show Cancellation, Active Trip Telemetry, GPS Position Stream over WebSocket.

Test Case ID	Test Title	Description	Preconditions	Test Steps	Test Data	Expected Output	Actual Output	Status
TC_TM_01	Verify driver updates status to EN_ROUTE	Ensure driver advances trip state to EN_ROUTE when heading to pickup point.	1. Driver accepted Trip #101 (status ACCEPTED)	1. Driver opens active ride view 2. Click 'Start Navigation to Pickup' 3. Calls PUT /api/trips/101/status	Trip ID: 101 Status: EN_ROUTE	HTTP 200 OK; trip status updated to EN_ROUTE in MSSQL; passenger notified.	HTTP 200 OK; status updated to EN_ROUTE; message 'Trip status updated to EN_ROUTE'.	**Pass**
TC_TM_02	Verify driver updates status to ARRIVED at pickup	Ensure driver marks arrival at pickup and starts passenger wait timer.	1. Trip #101 status is EN_ROUTE	1. Driver reaches pickup location 2. Click 'I Have Arrived' 3. Calls PUT /api/trips/101/status	Trip ID: 101 Status: ARRIVED	Trip status updated to ARRIVED; arrival timestamp logged; 5-min wait countdown starts.	HTTP 200 OK; status set to ARRIVED in trips table; passenger notification sent.	**Pass**
TC_TM_03	Verify driver starts trip - status IN_PROGRESS	Ensure driver initiates ride once passenger is onboard.	1. Trip #101 status is ARRIVED 2. Passenger has boarded vehicle	1. Driver confirms passenger boarding 2. Click 'Start Trip' 3. Calls PUT /api/trips/101/status	Trip ID: 101 Status: IN_PROGRESS	Trip status updated to IN_PROGRESS; trip start timestamp recorded.	HTTP 200 OK; status updated to IN_PROGRESS in MSSQL; live trip timer started.	**Pass**
TC_TM_04	Verify driver completes trip - status COMPLETED	Ensure driver marks trip complete upon reaching destination.	1. Trip #101 status is IN_PROGRESS 2. Vehicle reached dropoff location	1. Driver arrives at destination 2. Click 'Complete Trip' 3. Calls PUT /api/trips/101/status	Trip ID: 101 Status: COMPLETED	Trip status becomes COMPLETED; completed_at timestamp recorded; ready for payment.	HTTP 200 OK; status set to COMPLETED; completed_at timestamp updated in MSSQL.	**Pass**
TC_TM_05	Verify illegal state transition skip (EN_ROUTE to COMPLETED)	Ensure state machine enforces sequential transitions and blocks skipped states.	1. Trip #101 is in EN_ROUTE status	1. Driver attempts to send status COMPLETED without passing ARRIVED and IN_PROGRESS	Trip ID: 101 Target Status: COMPLETED	Request rejected with HTTP 409 Conflict: invalid state sequence.	Rejected (HTTP 409): 'Illegal state transition from EN_ROUTE to COMPLETED. Must be IN_PROGRESS first.'	**Pass**
TC_TM_06	Verify driver no-show cancellation after 5 minutes wait	Ensure driver can cancel with LKR 100 fee after waiting >= 5 mins at pickup.	1. Trip #101 status is ARRIVED 2. Driver wait elapsed >= 300 seconds (5 mins)	1. Wait countdown expires 2. Driver clicks 'Cancel No-Show' 3. Calls POST /api/trips/101/noshow	Trip ID: 101 Wait Duration: 310 seconds	Trip cancelled; LKR 100 no-show fee applied; driver released for new rides; HTTP 200.	HTTP 200 OK; cancelled=true; noShowFee=100.00; fee logged to passenger.	**Pass**

TC_TM_07	Verify driver no-show cancellation before 5 minutes wait	Boundary test: ensure no-show cancellation is blocked if wait time < 5 mins.	1. Trip #101 status is ARRIVED 2. Driver wait elapsed is only 120 seconds	1. Driver clicks 'Cancel No-Show' prematurely	Trip ID: 101 Wait Duration: 120 seconds (< 300s)	Action rejected: driver must wait at least 5 minutes before applying no-show.	Rejected (HTTP 400): 'Cannot report no-show. Driver must wait at least 5 minutes after arrival.'	**Pass**
TC_TM_08	Verify driver fetching active current trip	Ensure driver dashboard restores active trip upon screen mount.	1. Driver driver1@streetify.com has ongoing trip #101	1. Driver opens driver dashboard 2. Calls GET /api/trips/current	Driver Bearer JWT token	HTTP 200 OK; current trip payload returned with passenger and route details.	HTTP 200 OK; active trip details returned with status IN_PROGRESS; map view restored.	**Pass**
TC_TM_09	Verify live GPS telemetry broadcast over WebSocket	Ensure real-time driver coordinates stream to passenger map channel.	1. Driver connected to STOMP WebSocket broker at /ws 2. Active trip #101	1. Driver sends GPS payload to destination /telemetry 2. Check broker delivery to passenger queue	Payload: { tripId: 101, lat: 6.9147, lng: 79.9729, speed: 45.0, heading: 90.0 }	Message delivered to /user/{passengerEmail}/queue/trip/101 and /topic/fleet.	Telemetry broadcasted successfully; passenger client receives position and updates car marker.	**Pass**
TC_TM_10	Verify unauthorized driver cannot update another driver's trip	Security check: ensure a driver cannot update status on an unassigned trip.	1. Trip #101 assigned to Driver 1 2. Driver 2 is logged in	1. Driver 2 attempts to send PUT /api/trips/101/status	Trip ID: 101 Caller: Driver 2	Action rejected with HTTP 403 Forbidden: caller not assigned to this trip.	Refused (HTTP 403): 'You are not authorized to update status for this trip.'	**Pass**

4. Payment and Receipt Management

Member: E.A.R.N.D. Edirisinghe (Daham) | **Registration No.: IT25102225**

Scope: Payment Settlement (CASH, CARD, WALLET), Digital Wallet Deduction, 15% Platform Commission & 85% Driver Net Calculation, Duplicate Payment Prevention, Driver Commission Debt, Receipt Generation.

Test Case ID	Test Title	Description	Preconditions	Test Steps	Test Data	Expected Output	Actual Output	Status
TC_PM_01	Verify digital wallet payment with sufficient balance	Ensure passenger wallet is debited, driver credited (85%), and platform keeps commission (15%).	1. Trip #101 is COMPLETED, total fare LKR 1000.00, unpaid 2. Passenger wallet balance is LKR 5000.00	1. Open Payment screen 2. Select 'WALLET' method 3. Click 'Pay Now' 4. Calls POST /api/payments/process	Trip ID: 101 Payment Method: WALLET Fare: LKR 1000.00	Payment SUCCESS; passenger debited LKR 1000; driver credited LKR 850; platform commission LKR 150.	Payment SUCCESS (HTTP 200); passenger wallet now LKR 4000.00; driver wallet credited LKR 850.00; receipt generated.	**Pass**
TC_PM_02	Verify digital wallet payment with insufficient balance	Ensure wallet payment is rejected when balance is lower than total fare.	1. Trip #102 fare is LKR 1500.00 2. Passenger wallet balance is LKR 250.00	1. Select 'WALLET' method 2. Click 'Pay Now'	Trip ID: 102 Payment Method: WALLET Required: LKR 1500.00 Balance: LKR 250.00	Payment rejected with HTTP 409 Conflict; message explains insufficient funds.	Rejected (HTTP 409): 'Insufficient wallet balance (Current: LKR 250.00, Required: LKR 1500.00). Top up or pay via Cash/Card.'	**Pass**
TC_PM_03	Verify cash payment records 15% platform commission debt to driver	Ensure cash payment records commission debt to driver without debiting passenger wallet.	1. Trip #103 fare is LKR 2000.00 2. Driver commission debt is LKR 0.00	1. Select 'CASH' payment method 2. Process payment settlement	Trip ID: 103 Payment Method: CASH Fare: LKR 2000.00	Trip marked paid; driver commission_debt incremented by LKR 300.00 (15% of 2000); receipt issued.	HTTP 200 OK; status SUCCESS; driver commission_debt updated to LKR 300.00 in MSSQL; is_paid=1.	**Pass**
TC_PM_04	Verify card payment with masked card details and 3DS gateway simulation	Ensure online card payment succeeds and logs masked card info for compliance.	1. Trip #104 fare is LKR 1200.00 2. Unpaid completed trip	1. Select 'CARD' 2. Enter card details (4111 1111 1111 4321) 3. Click 'Pay Now'	Card: Visa **** 4321 Amount: LKR 1200.00 cardType: VISA	Payment succeeds; driver credited LKR 1020.00 (85%); masked card details saved.	HTTP 200 OK; status SUCCESS; card recorded as **** 4321; driver wallet credited LKR 1020.00.	**Pass**
TC_PM_05	Verify paying for an already paid trip (duplicate payment prevention)	Ensure system prevents charging an already paid trip.	1. Trip #101 is already paid (is_paid = 1)	1. Attempt to resubmit payment for Trip #101	Trip ID: 101 Payment Method: CARD	Payment rejected with HTTP 409 Conflict: trip already paid.	Rejected (HTTP 409): 'This trip has already been paid.' No double charge executed.	**Pass**
TC_PM_06	Verify payment gateway retry loop exhaustion on decline	Ensure payment retry loop runs up to 3 attempts before marking FAILED.	1. Simulated payment gateway configured to return failure	1. Process card payment with declining gateway response	Trip ID: 105 Method: CARD (Simulated gateway failure)	Gateway retried 3 times; payment status marked FAILED; trip remains unpaid.	3 retries logged in DB (retryCount: 3); receipt returned with status FAILED; is_paid remained 0.	**Pass**

TC_PM_07	Verify payment request with empty payment method	Ensure payment validation requires valid payment method parameter.	1. Trip completed and unpaid	1. Send payment request with paymentMethod: ''	Trip ID: 106 paymentMethod: ''	Rejected with HTTP 400 Bad Request; payment method is required.	Rejected (HTTP 400) on paymentMethod: 'Payment method is required'.	**Pass**
TC_PM_08	Verify exact zero wallet balance boundary test	Boundary test: passenger wallet reaches exactly LKR 0.00 without negative balance.	1. Passenger wallet balance is exactly LKR 1000.00 2. Trip fare is exactly LKR 1000.00	1. Pay full fare using 'WALLET' method	Fare: LKR 1000.00 Wallet Balance: LKR 1000.00	Payment succeeds; new wallet balance in MSSQL is exactly LKR 0.00; no overdraft.	Payment SUCCESS (HTTP 200); passenger wallet balance updated to 0.00; transaction logged.	**Pass**
TC_PM_09	Verify passenger cannot pay for another user's trip	Security check: ensure only the trip owner can execute payment.	1. Trip #107 belongs to Passenger A 2. Passenger B is authenticated	1. Passenger B attempts to execute payment for Trip #107	Trip ID: 107 (Passenger A) Caller: Passenger B	Payment rejected with HTTP 403 Forbidden: unauthorized caller.	Refused (HTTP 403) with 'You are not authorized to pay for this trip.'	**Pass**
TC_PM_10	Verify payment receipt retrieval and financial audit fields	Ensure generated receipt contains complete financial audit ledger fields.	1. Trip #101 payment completed successfully	1. Passenger opens receipt view on ReceiptDisplay.tsx	Payment ID: 501	Receipt renders paymentId, tripId, grossAmount, commission, driverNet, and timestamp.	All financial ledger fields rendered accurately; values match MSSQL payments table.	**Pass**

5. Review, Rating, and Feedback Management

Member: W.A.M.B. Weerasingha (Mithun) | **Registration No.: IT25102193**

Scope: Post-Trip Review Submission, 1-to-5 Star Rating Boundary Validations, Driver Average Rating Aggregation, Duplicate Review Prevention, Dispute Ticket Submission, Support Staff Ticket Resolution.

Test Case ID	Test Title	Description	Preconditions	Test Steps	Test Data	Expected Output	Actual Output	Status
TC_RM_01	Verify submitting 5-star review for completed trip	Ensure passenger can submit valid 5-star review and update driver rating.	1. Trip #101 is COMPLETED and paid 2. Passenger is logged in	1. Open Review screen 2. Select 5 stars 3. Enter feedback comment 4. Click 'Submit Review'	Trip ID: 101 Rating: 5 Comment: 'Smooth ride and excellent driving!'	HTTP 201 Created; review saved; driver average rating recalculated in MSSQL.	HTTP 201 Created; review record #201 saved; driver average_rating updated to 4.9.	**Pass**
TC_RM_02	Verify star rating boundary minimum (1 Star allowed)	Boundary test: verify lowest permissible rating (1 star) is accepted.	1. Trip #102 completed and paid	1. Select 1 star 2. Enter comment: 'Driver took wrong route' 3. Click 'Submit Review'	Trip ID: 102 Rating: 1 Comment: 'Driver took wrong route'	Review saved; 1-star rating accepted; driver average rating recalculated.	HTTP 201 Created; 1-star review saved in reviews table; driver average rating adjusted.	**Pass**
TC_RM_03	Verify star rating below minimum (0 Stars rejected)	Boundary test: ensure rating < 1 is rejected by bean validation.	1. Trip completed and paid	1. Send review payload with rating: 0	Trip ID: 103 Rating: 0 Comment: 'Terrible'	Rejected with HTTP 400 Bad Request; rating must be at least 1 star.	Rejected (HTTP 400) on rating: 'Rating must be at least 1 star' (@Min(1) violation).	**Pass**
TC_RM_04	Verify star rating above maximum (6 Stars rejected)	Boundary test: ensure rating > 5 is rejected by bean validation.	1. Trip completed and paid	1. Send review payload with rating: 6	Trip ID: 103 Rating: 6 Comment: 'Beyond excellent'	Rejected with HTTP 400 Bad Request; rating cannot exceed 5 stars.	Rejected (HTTP 400) on rating: 'Rating cannot exceed 5 stars' (@Max(5) violation).	**Pass**
TC_RM_05	Verify duplicate review submission on same trip	Ensure passenger cannot review the same trip more than once.	1. Review already submitted for Trip #101	1. Passenger attempts to submit another review for Trip #101	Trip ID: 101 Rating: 4 Comment: 'Updating feedback'	Request rejected with HTTP 409 Conflict: trip already reviewed.	Rejected (HTTP 409): 'You have already submitted a review for this trip.'	**Pass**
TC_RM_06	Verify review submission on incomplete trip	Ensure review can only be submitted for COMPLETED trips.	1. Trip #104 status is IN_PROGRESS	1. Passenger attempts to submit review for ongoing Trip #104	Trip ID: 104 Rating: 5	Rejected: cannot review a trip that is not COMPLETED.	Rejected (HTTP 400): 'Cannot review a trip that is not COMPLETED.'	**Pass**
TC_RM_07	Verify dispute ticket creation with valid details	Ensure passenger can file support dispute for an overcharged/problematic trip.	1. Passenger logged in 2. Trip #101 is completed	1. Open Support screen 2. Select dispute type 'OVERCHARGED' 3. Enter subject and description 4. Submit dispute ticket	Trip ID: 101 Type: OVERCHARGED Subject: Route Detour Overcharge Description: Driver took 5km detour.	HTTP 201 Created; dispute ticket saved with status OPEN; reference generated.	Dispute ticket created (HTTP 201); status OPEN; reference TKT-00101 logged in MSSQL.	**Pass**

TC_RM_08	Verify dispute ticket submission without description	Ensure dispute form requires detailed explanation.	1. Passenger on Dispute form	1. Enter subject 2. Leave description empty 3. Click 'Submit Dispute'	Subject: Overcharged Description: '' (empty)	Validation rejected with HTTP 400; description is required.	Rejected (HTTP 400) on description: 'Description is required' (@NotBlank).	**Pass**
TC_RM_09	Verify support officer resolving dispute with wallet refund	Ensure support staff can resolve ticket and credit passenger refund.	1. Dispute ticket #301 is OPEN 2. Logged in as Support Staff (admin@streetify.com)	1. Review ticket #301 2. Set decision RESOLVED 3. Enter refund amount LKR 300.00 4. Submit resolution	Ticket ID: 301 Decision: RESOLVED Refund Amount: LKR 300.00 Resolution Note: Detour verified.	Ticket updated to RESOLVED; passenger wallet credited LKR 300 in MSSQL; HTTP 200.	HTTP 200 OK; ticket status RESOLVED; passenger wallet increased by LKR 300.00.	**Pass**
TC_RM_10	Verify passenger cannot access staff dispute queue	Role check: ensure passenger cannot view global pending support tickets.	1. Passenger is authenticated (role PASSENGER)	1. Passenger calls GET /api/disputes/pending	Caller: passenger1@streetify.com	Access denied with HTTP 403 Forbidden.	Refused (HTTP 403): 'You do not have permission to perform this action.'	**Pass**

6. Administration and System Control

Member: R.V.S. Rammandalagedara (Vidura) | **Registration No.: IT25102240**

Scope: Role-Based Access Control (RBAC), Admin Dashboard Live KPIs, Platform Revenue Audits, Driver Document Verification Queue, Account Suspension & Audit Trail, Conceptual JWT Revocation, Cross-Module Data Governance.

Test Case ID	Test Title	Description	Preconditions	Test Steps	Test Data	Expected Output	Actual Output	Status
TC_AM_01	Verify unauthorized user denied access to admin control endpoints	RBAC security test: non-admin users must be rejected from administrative routes.	1. Authenticated as regular PASSENGER or DRIVER	1. Attempt to call GET /api/admin/stats or GET /api/module-admin/users directly	Caller: passenger1@streetify.com Role: PASSENGER	HTTP 403 Forbidden; access denied; no administrative data exposed.	Refused (HTTP 403): 'You do not have permission to perform this action.'	**Pass**
TC_AM_02	Verify admin can access live dashboard KPIs and metrics	Ensure admin dashboard aggregates real-time platform statistics accurately.	1. Logged in as ADMIN (vidura@streetify.lk)	1. Open Admin Dashboard 2. System calls GET /api/admin/stats	Admin Bearer JWT token	HTTP 200 OK; returns total users, active drivers, completed trips, revenue, disputes.	HTTP 200 OK; KPIs returned matching database aggregates in MSSQL.	**Pass**
TC_AM_03	Verify daily platform commission revenue calculation	Ensure platform revenue endpoint sums 15% commission from completed trips.	1. Admin logged in 2. Completed trips exist for current date	1. Call GET /api/admin/revenue/daily	Date: Current Date Admin JWT token	HTTP 200 OK; returns total revenue in LKR matching database sum.	HTTP 200 OK; revenue returned matching sum of platform_commission column in trips table.	**Pass**
TC_AM_04	Verify admin reviews pending driver document queue	Ensure admin can inspect driver license, CR book, and insurance uploads.	1. Driver 3 (driver3@streetify.com) has uploaded documents (status PENDING)	1. Admin opens Document Verification Queue 2. Calls GET /api/admin/verifications	Admin Bearer token	HTTP 200 OK; pending documents for Driver 3 listed with file metadata.	HTTP 200 OK; 3 pending documents returned (license, reg, insurance) with file paths.	**Pass**
TC_AM_05	Verify admin approves driver documents and activates driver	Ensure admin approval updates driver status to APPROVED in database.	1. Driver 3 documents in PENDING status 2. Driver verification_status is PENDING_VERIFICATION	1. Admin reviews driver documents 2. Clicks 'Approve All Documents' 3. Approves driver profile	Driver ID: 12 Reviewer: Vidura (Admin) Status: APPROVED	Driver documents set to APPROVED; driver verification_status set to APPROVED; driver activated.	Status updated to APPROVED in MSSQL; driver allowed to toggle online; audit log logged.	**Pass**
TC_AM_06	Verify admin suspends user account with mandatory audit note	Ensure admin can suspend fraudulent/violating accounts and write audit trail.	1. Target user exists (User ID: 15) 2. Admin logged in	1. Open Suspend User modal 2. Enter duration: '7 days' 3. Enter audit note: 'Safety violation reported' 4. Click 'Suspend Account'	userId: 15 duration: '7 days' auditNote: 'Safety violation reported'	User suspended flag set to 1 in MSSQL; immutable audit record written; HTTP 200.	User suspended=1 in users table; audit log #801 created with action ACCOUNT_SUSPENDED.	**Pass**

TC_AM_07	Verify suspending user without audit note is rejected	Ensure accountability by requiring non-blank audit note for suspensions.	1. Admin on Suspend User modal	1. Select target user 2. Leave auditNote blank 3. Click 'Suspend Account'	userId: 15 auditNote: '' (blank)	Rejected with HTTP 400 Bad Request; audit note is mandatory (@NotBlank).	Rejected (HTTP 400) on auditNote: 'Audit note is required for account suspension'.	**Pass**
TC_AM_08	Verify admin lifts user suspension (unsuspend account)	Ensure admin can reinstate suspended user and log restoration in audit trail.	1. User ID: 15 is currently suspended (suspended = 1)	1. Admin clicks 'Unsuspend Account' 2. Enters resolution note 3. Calls POST /api/admin/unsuspend/15	userId: 15 note: 'Appeal approved after investigation'	User suspended flag reset to 0 in MSSQL; audit record UNSUSPEND_ACCOUNT logged; user can log in.	HTTP 200 OK; user suspended=0; audit log written; message 'Suspension lifted for user 15'.	**Pass**
TC_AM_09	Verify conceptual JWT session revocation broadcast	Ensure admin session revocation broadcasts disconnect event to user.	1. Target user has active JWT session	1. Admin triggers JWT revocation 2. Calls POST /api/admin/revoke-jwt?userId=15	userId: 15 Admin token	Audit log JWT_REVOKED created; WebSocket disconnect dispatched to target user.	HTTP 200 OK; revokedCount: 1; audit log saved; client session invalidated.	**Pass**
TC_AM_10	Verify immutable audit log retrieval with boundary limit cap	Boundary test: verify limit parameter caps safely at maximum 200 entries.	1. Over 200 audit entries exist in MSSQL audit_logs table	1. Admin queries audit log with limit=500 2. Calls GET /api/admin/audit?limit=500	Requested limit: 500 Maximum safe cap: 200	System returns exactly 200 most recent audit entries sorted descending by timestamp.	Exactly 200 records returned; safeLimit capped; records sorted newest first.	**Pass**